PIM user manual

Start and basic use

From the pim folder, run python3 main.py. Enter one command per line at the pim> prompt. Enter help to display the command reference. The system starts empty unless launched with --file. Changes remain in memory until you save them.

```
python3 main.py
python3 main.py --file "my-records.pim"
python3 main.py --script examples/demo.txt
```

The demonstration creates four fictional records and writes/replaces demo.pim. The --script option reads a UTF-8 command file; blank lines and lines beginning with # are ignored. Search examples in this manual are entered at the PIM prompt, not directly in your operating system shell.

Create records

All fields shown for each type are required. Use field=value with no spaces around the assignment sign. Quote values containing spaces with double or single quotes. Use double quotes around text containing apostrophes. Fields may appear in any order. Duplicate, unknown and missing fields are rejected.

```
create note text="Project ideas"
create task description="Submit project" deadline=2026-11-20T20:00
create event description="Team meeting" start=2026-11-12T10:00 alarm=2026-11-12T09:45
create contact name="Chan Tai Man" address="Hung Hom, Hong Kong" mobile="+852 9123 4567"
```

Text values must contain a non-whitespace character. Original spaces and Unicode text are retained. Mobile numbers are text: leading zeros, country codes and spaces are preserved. No specific national phone-number format is imposed.

Times must include both date and time in ISO 8601 format, such as 2026-11-20T20:00 or 2026-11-20T20:00+08:00. A time with a space instead of T must be quoted. A missing timezone means Hong Kong (+08:00). Explicit offsets, including Z for UTC, are converted to +08:00. Equal instants in different offsets compare equal. An event alarm must be at or before the event's start. Alarms are stored and searchable timestamps; the program does not issue notifications.

Creation prints the assigned ID, type and every field. IDs are positive integers that identify records, remain stable on update/save/load, and are not reused after deletion within that collection. Loading another file switches to that file's collection and IDs.

View and change records

```
list
show 1
update 1 text="Revised project ideas"
update 2 description="Final submission" deadline=2026-11-20T19:00
delete 4
```

list prints every record, sorted by ID. show prints one record's complete information. search also prints complete information for all matches. Output begins with [ID] type, followed by field names and quoted values. Escaped sequences in the output, such as \n or \" within quoted text, represent stored newline characters or quotation marks. A result count of 0 record(s) is a valid empty result.

update changes only the specified fields, keeping the ID, type and other fields. At least one field must be supplied. Multiple changes are validated together; failure leaves the original record intact. ID and type cannot be

changed. delete removes one record immediately from memory; save persists that deletion. There is no undo operation.

Search expressions

```
search type = task
search text contains "project ideas"
search description contains "project"
search mobile contains "9123"
search deadline < "2026-11-21T00:00"
search start > "2026-11-12T09:00"
search alarm = "2026-11-12T01:45Z"
search (type = task || type = event) && description contains "project"
search !(type = contact) && (text contains "ideas" || description contains "project")
```

Field	Operator	Meaning
type	=	Exact type: note, task, event or contact
text, description, name, address, mobile	contains	Case-insensitive substring match
deadline, start, alarm	<, >, =	Strictly before, strictly after, or same instant

Field names, type names, commands and the contains operator are lowercase and case-sensitive. Only the text substring comparison ignores case. A field belongs only to the corresponding record type. Testing an absent field is false, so !(deadline < "2026-12-01T00:00") also matches records without a deadline. To search only tasks, combine that condition with type = task.

Use ! for logical negation, && for and, and || for or. Evaluation priority is parentheses first, then !, then &&, then ||. Add parentheses to make your intention clear. A chain such as type=note || type=task && description contains "due" means type=note || (type=task && description contains "due").

Search values with spaces or symbols must use JSON double-quoted strings, unlike the ordinary command field tokenizer. Within a quoted search string, use \" for a literal quotation mark and \\ for a literal backslash. A quoted empty search value matches every record containing that text field. Single-quoted search strings are not supported. Text entered in a create command is a single-line field value; the file format and model can also preserve multiline text.

Expressions may contain at most 4096 characters and at most 50 levels of nested parentheses/negations. Unsupported fields, operators, invalid dates, missing operands and mismatched parentheses produce an error instead of executing a partial search.

Save and load files

```
save "my-records.pim"
load "my-records.pim"
load "my-records.pim" --force
quit
quit --force
```

Paths may be absolute or relative to the directory where you launched the program. Quote paths containing spaces. The extension must be .pim; .PIM is accepted too. The parent directory must already exist. save writes the entire collection and replaces an existing destination. Data files use readable UTF-8 JSON, not encryption; store them in an appropriate personal location.

load replaces the entire current collection after completely validating the file. Unsaved changes block loading unless you save first or explicitly use --force to discard them. A failed load keeps current records and unsaved

state. An invalid or unsupported file is reported as an error. There is no record merging.

quit exits if there are no unsaved changes. With unsaved changes, it asks you through an error message to save first or use quit --force. That option deliberately discards unsaved work. There is no automatic saving. End-of-input or interruption also does not save; avoid closing the terminal before saving.

Errors and scripted operation

Invalid commands print ERROR and an explanation to standard error. Enter help to check syntax; the interactive session remains available after ordinary errors. A failed create/update does not alter records, and a failed save does not mark unsaved changes as saved. An atomic replacement failure preserves an existing destination file.

In script mode, processing continues after ordinary command errors so later lines can still run; the final exit status is 1 if any command failed. Status 0 means successful completion, 1 also covers startup/file errors and unsaved end-of-input, 2 is an invalid startup option, and 130 means keyboard interruption during a session. A clean interactive quit returns 0 even if an earlier command was corrected. The quit command stops processing any later script lines.

For a simple persisted session, create records, inspect or search them, run save with a .pim path, and then quit. Reopen that file with --file or load to continue.